

CS633 Assignment 2 : Group 57

Ch V Sai Koushik (230312)
Chilamakuri Kundan Sai (230330)
Challa Kethan (230317)
Kamatam Akshay Reddy (230517)
Kishore Senthilkumar (230566)

Date: 12/04/2026

1. Problem Overview

The program distributes a 3-D Cartesian domain across $P = p_x \times p_y \times p_z$ MPI ranks. Each rank owns a local block of $n_x \times n_y \times n_z$ grid points for each of F scalar fields. Over T time steps the program applies a d -point star stencil, exchanges halo data with neighbours, counts isovalue crossings per field and reduces those counts to rank 0, which prints F counts per step and the total wall-clock time at the end. The stencil radius is $h = (d - 1)/6$, giving $h = 1$ for $d = 7$. At the global domain boundary, points use only the neighbours that exist and divide by that count accordingly.

2. Code Design and Logic

Data Logic : The stencil is a 3-D *star* stencil, it uses only the **6 axis** aligned directions ($\pm x, \pm y, \pm z$) plus the centre point (not a full 27-point cube stencil). Accordingly, halo exchange covers only the 6 faces of each sub-domain, edges and corners are never communicated.

Each field is stored as a flat array of size $g_x \times g_y \times g_z$ ($g_x = n_x + 2h$, $g_y = n_y + 2h$, $g_z = n_z + 2h$) with h -wide ghost-cell layers on all six sides. Ghost layers hold a copy of the neighbouring rank's boundary data, so the stencil kernel never needs to distinguish between local and received values. All F fields are packed into a single contiguous allocation (**current_data**), and a second identical allocation (**next_data**) holds the values being written into. At the end of each step the two pointers are *swapped* (no data is copied).

At global domain boundaries (ranks with no neighbour on one or more sides), the stencil sums only the neighbours that exist and are divided by their count. **Isovalue counting is done on axis-aligned edges only** (x -, y -, z -edges between adjacent grid points), not diagonals. Edges that cross into a neighbouring rank's subdomain are included through the ghost cells, which means crossings at process boundaries are counted by both ranks (**double counting**).

Stepwise sequence flow : Counting for step k is done at the start of step $k+1$, after the halo exchange has already filled the ghost cells needed for correct boundary edge counting. Each iteration follows this order:

1. **Start halo on current_data:** post all existing face **MPI_Irecv**s first (so the network can deliver data immediately when ready), then pack the required face face slabs into send buffers and post all six **MPI_Isends**.
2. **Compute interior** of **next_data** using **current_data** (all points at distance $\geq h$ from every sub-domain face). This runs *concurrently* with the in-flight messages, overlapping communication with computation.
3. **Wait and unpack:** **MPI_Waitall** blocks until all messages arrive; received data is then unpacked directly into the ghost layers of **current_data**.
4. **Count and reduce** (skipped on step 1): **count_iso** counts isovalue crossings on **current_data** (which holds the result of the *previous* stencil step), then **MPI_Reduce** sums counts to rank 0, which prints the output line for that step.
5. **Compute boundary** of **next_data:** Update the six boundary slabs of **next_data** using the ghost-cell values that just arrived.
6. **Swap current_data** $\leftrightarrow$ **next_data** by pointer exchange.

This ordering ensures that ghost cells are always populated before any stencil read or isovalue count that depends on them. The count for step k is always printed at the start of step $k+1$, so the count for the last step T has no following iteration to trigger it. A single halo exchange after the loop fills the ghost cells and the final count is printed. Timing uses **MPI_Barrier** before **MPI_Wtime()** so all ranks start together, and reports the *maximum* elapsed time across all ranks via **MPI_Reduce(..., MPI_MAX, ...)**.

3. Code Optimizations

3.1 Communication - Computation Overlap

The most important optimization is overlapping halo communication with interior stencil computation. Receives are posted before sends so that communication is initiated as early as possible, and interior computation can proceed while halo messages are in flight. `MPI.Waitall` is called only after the interior update finishes, and the boundary slabs are computed only after the wait returns.

For $d = 7$ ($h = 1$) with a 120^3 local block, the interior region is 118^3 out of 120^3 total points — about **95%** of per-step work overlaps with communication. At 240^3 this is $\approx 96\%$. Since communication volume grows as $O(n^2)$ while compute grows as $O(n^3)$, this overlap becomes more effective as the subdomain grows.

3.2 Ghost-Cell Storage and Boundary Slab Split

Received halo values are unpacked into ghost layers that are part of the same allocated array as the local data, so the stencil reads local and ghost-cell neighbours identically, with no per-point boundary check inside the loop. The update is split into `compute_interior` and `compute_boundary`, the boundary function calls `update_box` for six thin face slabs only, processing just the $\approx 5\%$ of points that need ghost data (the remaining 95% is handled by `compute_interior`), instead of scanning all points with per-point boundary checks.

3.3 Precomputed Valid-Neighbour Counts

Rather than checking at every step whether a boundary point has a left neighbour, a right neighbour, and so on, `build_valid_counts()` computes the results once at startup, before any time step runs. The answers are stored in six small integer arrays. The boundary kernel simply reads the precomputed count for each direction, loops exactly that many times, and sums the results without any branches or repeated logic.

3.4 Contiguous Storage and Pointer Walking

Two strides (`step_x`, `step_y`) are precomputed once outside all loops. In the innermost z -loop of `compute_interior`, source and destination pointers advance with `p++` and `q++` each iteration, all six neighbors are reached via stride offsets with no index recomputation. All F fields share one contiguous allocation, so switching between fields does not jump to a separate memory region. The two time-step buffers are advanced by swapping pointers rather than copying.

4. Results

All runs used the parameter values of $d = 7$, $T = 5$, seed= 1000, $F = 2$, isovalue= 500 with each configuration run 5 independent times. The process grid mappings were:

$$\begin{aligned} P = 32 : (px, py, pz) &= (4, 4, 2), \text{ } ppn = 32 & P = 48 : (px, py, pz) &= (6, 4, 2), \text{ } ppn = 48 \\ P = 64 : (px, py, pz) &= (4, 4, 4), \text{ } ppn = 32 & P = 96 : (px, py, pz) &= (6, 4, 4), \text{ } ppn = 48 \end{aligned}$$

From the SLURM job output, $P = 32$ ran on a single node and $P = 48$ on a single node, while $P = 64$ and $P = 96$ each ran across two nodes.

Wall clock time (sec); $nx = ny = nz = 120$

P	Node(s)	Avg	R1	R2	R3	R4	R5
32	1 (rdcn25)	0.5623	0.5568	0.5627	0.5662	0.5612	0.5645
48	1 (rdcn27)	0.5540	0.5554	0.5542	0.5518	0.5567	0.5520
64	2 (rdcn30-31)	0.5623	0.5611	0.5590	0.5660	0.5590	0.5662
96	2 (rdcn35-36)	0.5761	0.5686	0.5906	0.5737	0.5729	0.5748

Wall clock time (sec); $nx = ny = nz = 240$

P	Node(s)	Avg	R1	R2	R3	R4	R5
32	1 (rdcn25)	4.8822	4.8514	4.8765	4.9118	4.8835	4.8877
48	1 (rdcn27)	4.7900	4.7857	4.7637	4.7905	4.8107	4.7995
64	2 (rdcn35-36)	4.4646	4.4290	4.4466	4.4832	4.4727	4.4916
96	2 (rdcn30-31)	4.7331	4.7298	4.7245	4.7295	4.7421	4.7397

Assignment 2 - MPI Stencil Timing (d=7, T=5, F=2)

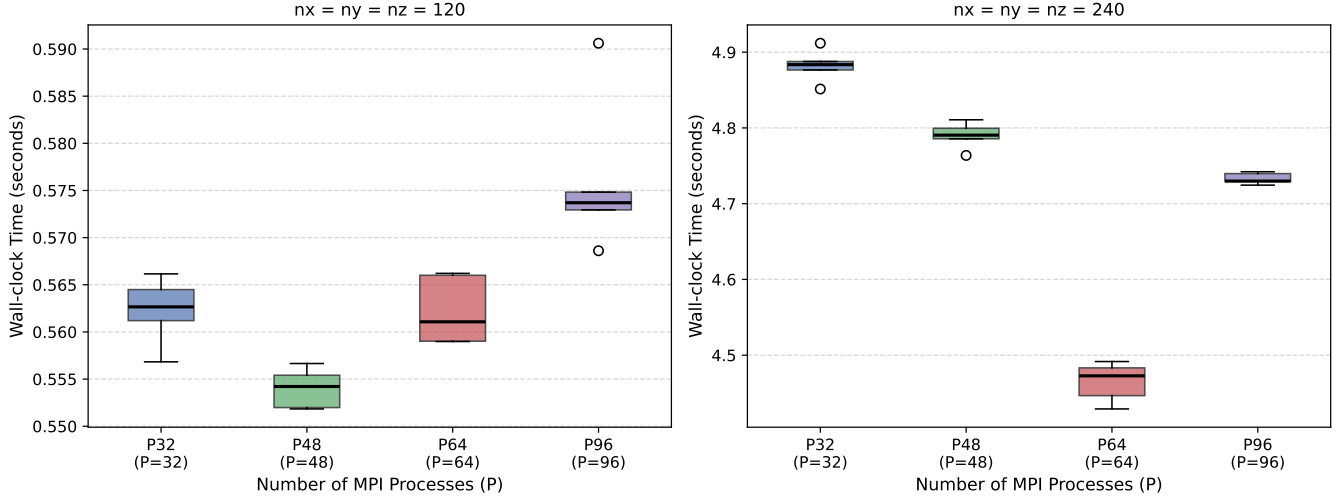


Figure 1: Boxplots of wall-clock time over 5 runs. Outlier runs are shown as open circles.

Effect of subdomain size :

Doubling the linear subdomain size from 120 to 240 makes the per-rank local domain $2^3 = 8$ times larger. The measured timing ratios across the four configurations are $8.68\times$, $8.65\times$, $7.94\times$, and $8.22\times$ for $P = 32, 48, 64, 96$, all very close to the expected $8\times$. This shows that the program scales cleanly with problem size and also dominated by the computation. We can also note the fact that communication volume grows by only $4\times$ (face area scales as n^2), so the 240^3 case is proportionally less communication bound.

Effect of process count and node topology :

For $n = 120$, the two single node configurations ($P = 32$ and $P = 48$) and the two-node $P = 64$ all achieve very similar average times. $P = 48$ on a single node is slightly fastest here. At this small subdomain size, the compute and communication load are less, and intra-node communication for $P = 48$ costs very little with the work load being less than $P = 32$. $P = 96$ across two nodes is the slowest as more processes with inter node messages for a problem that is small enough that the overhead is not hidden.

For $n = 240$, the results are more interesting. $P = 64$ across two nodes is faster than all single node configurations, even though per rank computation work is identical. At this larger subdomain size (240^3), having 32 processes on one node means 32 ranks simultaneously streaming through memory. Spreading to two nodes halves this per-node load, which is why $P = 64$ (32 processes per node) outperforms $P = 32$ and $P = 48$ (both on a single node). $P = 96$ across two nodes is faster than the single-node configs but slower than $P = 64$, because the higher process count adds more inter-node communication overhead than the bandwidth relief is worth at 48 processes per node.

Isovalue crossing counts :

All five runs of any given configuration produce identical values, since the computation is a fixed function of the input parameters. The two tables below show the total crossings per step (Field 1 + Field 2) for each configuration separately.

Isovalue crossings per step; $nx = ny = nz = 120$

P	(px, py, pz)	Step 1	Step 2	Step 3	Step 4	Step 5
32	(4,4,2)	625,616	139,327	30,658	6,352	1,144
48	(6,4,2)	624,970	139,153	30,594	6,348	1,144
64	(4,4,4)	625,616	139,327	30,658	6,352	1,144
96	(6,4,4)	624,970	139,153	30,594	6,348	1,144

All four configurations start around 625,000 crossings at step 1 and all four reach *exactly* 1,144 by step 5, a reduction of over $500\times$. $P = 32$ and $P = 64$ produce identical counts at every step ($625,616 \rightarrow 1,144$). This is because they share the same $px = 4$, $py = 4$ decomposition; only pz differs (2 vs 4), and the per-rank data is determined by **myrank** following the same x - y layout. Similarly, $P = 48$ and $P = 96$ (both with $px = 6$, $py = 4$) are identical to each other at every step, starting at 624,970. The slight difference between the two pairs (e.g. 625,616 vs 624,970 at step 1) comes from the different px values, which changes how **myrank** is assigned in x , producing slightly different initial data per rank.

Isovalue crossings per step; $nx = ny = nz = 240$

P	(px, py, pz)	Step 1	Step 2	Step 3	Step 4	Step 5
32	(4,4,2)	66,383,768	31,203,042	18,793,684	12,934,839	9,756,601
48	(6,4,2)	66,509,239	31,392,749	19,023,215	13,139,803	9,951,554
64	(4,4,4)	66,383,758	31,203,042	18,793,684	12,934,839	9,756,601
96	(6,4,4)	66,509,090	31,392,749	19,023,215	13,139,803	9,951,554

The counts are much larger in absolute terms (~ 66 million at step 1 vs $\sim 625,000$). $P = 32$ and $P = 64$ are again matched: their step 1 counts differ by just 10, and from step 2 onwards they are *identical*. $P = 48$ and $P = 96$ differ by 149 at step 1, then become identical from step 2. Across all four configs the decay is far slower. The larger domain takes many more steps to homogenise because the stencil averaging spreads diffusively.

5. Conclusion

The implementation distributes the 3-D stencil problem across a process grid, performs halo exchange, stencil update, and isovalue counting at every step, and prints results at rank 0. The central design follows overlapping non-blocking halo communication with interior stencil computation. This is implemented by ghost-cell storage for branch-free stencil access, precomputed valid-neighbour tables for clean boundary updates, and pointer-based inner loops over contiguous fields.

6. Contributions

Team Member Contributions

Roll No.	Name	Contribution
230312	Ch V Sai Koushik	20%
230317	Challa Kethan	20%
230330	Chilamakuri Kundan Sai	20%
230517	Kamatam Akshay Reddy	20%
230566	Kishore Senthilkumar	20%